

MSE 312: MECHATRONICS DESIGN II**Integration Lab 3: DC Motor Drive using Pulse Width Modulation (PWM)****OBJECTIVES**

To learn how Matlab/Simulink and Arduino embedded target board can be used to drive a DC Motor using PWM

Required Components/Material:

- Matlab/Simulink 2021b
- MEGA 2560 R3 Board ATmega2560 ATMEGA16U2 + USB Cable for Arduino
- Your PWM Drive Board built (using SN7400N and TB6568KQ ICs) in the electronics part of the course (in this lab we use a L298N H-bridge as an example), GM8724S009 DC Servo Gearmotor, Breadboard, 1 k Ω resistors, Arduino Jumper Cables, 10 k Ω potentiometer

References:

- Arduino Support from MATLAB/Simulink (www.mathworks.com)
- Rensselaer Arduino Support Package Library (RASPLib)
<https://www.mathworks.com/matlabcentral/fileexchange/62702-reusselaer-arduino-support-package-library-rasplib>

PROCEDURE**ACTIVITY 3.1: Rensselaer Arduino Support Package - RASPLib Installation (NOT NEEDED IF YOU ALREADY DID IT IN LAB 2)**

Rensselaer Polytechnic Institute has developed blocks that may be used for motor control including PWM waveform generation and reading rotary quadrature encoders. To use these blocks, the following steps should be followed. A folder named “RASPLib” and a script named “startup.m” will be provided to you on canvas.sfu.ca:

1. Copy and paste “startup.m” and “RASPLib” to your Matlab working directory, for example a directory on DESKTOP where your Simulink files are.
2. Run Matlab as administrator (right click on icon and choose “run as administrator”), then open and Run “startup.m” to add the directories to Matlab path. This will cause the Rensselaer Arduino Support Package entry to be added to the Simulink library browser as illustrated in the figure below. This process may take a while for the first time.

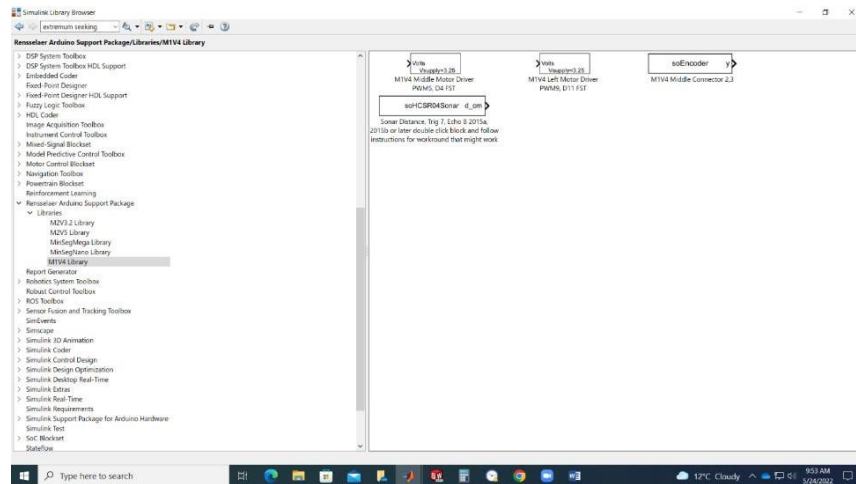


Figure 3.1: Rensselaer Arduino Support Package Simulink blocks.

3. Open the demo file for your particular hardware. The examples for the Arduino hardware is under the M1V4 Mega option.
4. After opening the Demo, save it to your home directory with “save-as”. From now on, you can create Simulink diagrams in any location and by dragging blocks from the library.

ACTIVITY 3.2: PWM signal Generation

Example 3.1: Dimming LED using PWM and Analog Input in Matlab

To control the brightness of an LED by changing the duty cycle of a pulse in MATLAB,

1. Select a digital output pin selectable for PWM from the selectable pins list provided in the Arduino’s datasheet (e.g., pins 2-13, 44-46) and connect it to the LED. Connect the rest of the circuit using a 1 k Ω resistor.
2. Select an analogue input and connect it to a potentiometer.
3. Run the following code:

```
time = 200;
while time > 0
    voltage = readVoltage(a, 'A0');
    writePWMPWMVoltage(a, 'D11', voltage);
    time = time - 0.1;
    pause(0.1);
end
```

As you run this code, by rotating the potentiometer knob, you can change the PWM voltage and see the result, i.e., dimming the LED.

Exercise

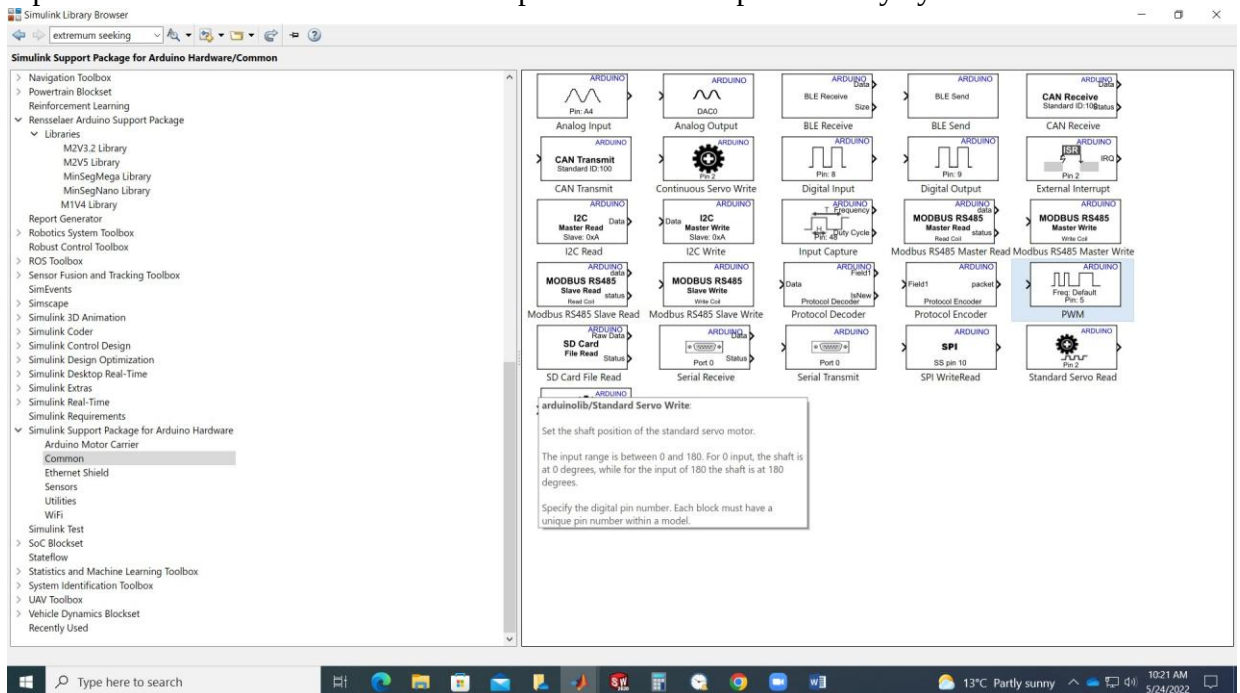
What does the following code do? Compare the results with the code above.

```
time = 100; dc=0;
while time > 0
    writePWMDutyCycle(a, 'D11', dc);
    % dc is the duty cycle: 0, 0.1, 0.4, ..., 1 (max)
    dc = dc+0.01;
    time = time - 0.1;
    pause(0.1);
end
```

Note: Remember to “clear” your Matlab workspace before moving to Simulink.

Example 3.2: Write PWM output (LED dimming) by using the Simulink Dashboard elements (slider)

1. Similar to step one of the previous example, select a digital output pin selectable for PWM and connect it to the + pin of the LED. Connect the - pin of the LED to the GND via a 1k resistor.
2. Drag and drop the “Arduino PWM” block from *Simulink Support Package for Arduino Hardware*, a “Constant” and a “Gain” from *Commonly Used Blocks* and a “Slider” from *Dashboard*. Connect the constant to the gain block and the gain block to the PWM block (see Figure 2.2). The PWM block creates a square waveform on the output pin. It takes an input value between 0 and 255 to correspond to 0 to 100 percent duty cycle.



3. Connect the slider to the gain block by following these steps:
 - a. Double click on the slider block to see the block parameters window
 - b. Select the gain block in the simulink editor to connect the tunable gain parameter to the slider. The gain parameter will appear in a list view.
 - c. Click on the gain block to see the gain as a parameter to tune in the block parameters
 - d. Click on the connect on the left side of the appeared gain variable.
 - e. Set the minimum and maximum of the slider to 0 and 255, respectively.

Connecting the slider to the gain allows you to change the duty cycle of the generated square waveform while the simulation is running.
4. Double click on the digital input block, then you can configure the digital input pin number to the one you want to read from (e.g., pin D8).

5. Use the **Model configuration instructions** Appendix to configure the model to be deployed on the Arduino hardware and run the simulation. deployed on the Arduino Hardware and start the simulation.
6. Examine what happens by changing the gain.
7. Use an oscilloscope to measure the duty cycle, pulse-width and the DC Voltage output of the created waveform at some sample gain values. Calculate the PWM frequency.
8. Create another slider such that the values ranging from 0 to 100 would generate duty cycles in the range 0 to 100%.

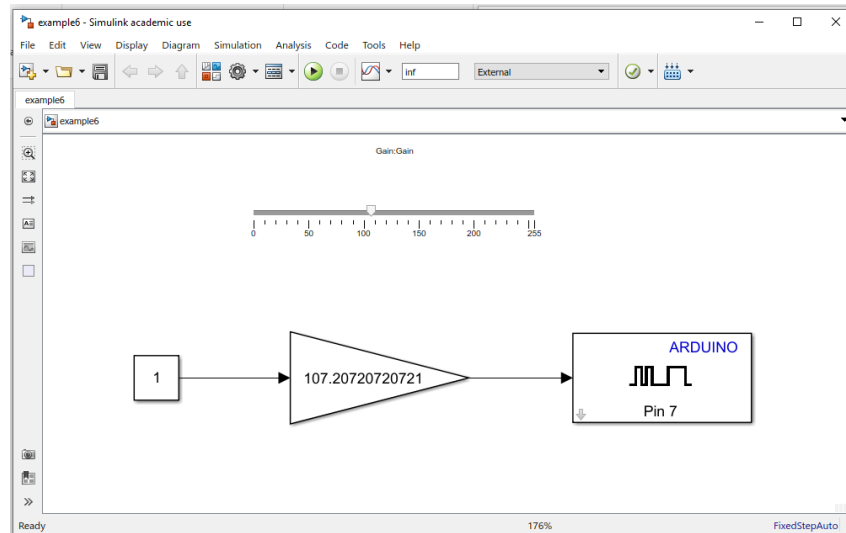


Figure 3.2: Write PWM output

Example 3.3: Changing PWM Frequency in Simulink

By default, the Mega 2560 Arduino, uses timer 0 to create PWM on pins 4-13 and the PWM frequency for these pins can be obtained from the following formula:

$$f_{PWM} = \frac{f_{clk} = 16MHz}{(\text{Prescaler} = 64) \times (\text{PWM mode number of states} = 256)} = 976.56 \text{ Hz}$$

For the rest of the PWM pins, timers 1 to 5 are used to generate PWM signal on these pins with the following frequency

$$f_{PWM} = \frac{f_{clk} = 16MHz}{64 \times 510} = 490.20 \text{ Hz}$$

When using pins that generate PWM signal with timers other than timer 0, the “Mega2560 PWM Frequency Select” block from *Rensselaer Arduino Support Package* can be used to

Note: The Rensselaer Arduino Support Package should already be installed. If it is not installed on your system, please return to the first page of this manual and follow the installation instructions before proceeding.

change the prescaler and change the PWM frequency. Frequencies 32kHz, 4kHz, 490Hz, 122Hz, 30Hz correspond to prescalers 1, 8, 64, 256, 1024, respectively.

Note that due to some issues with the third party support package, this block may only work in the 32kHz mode.

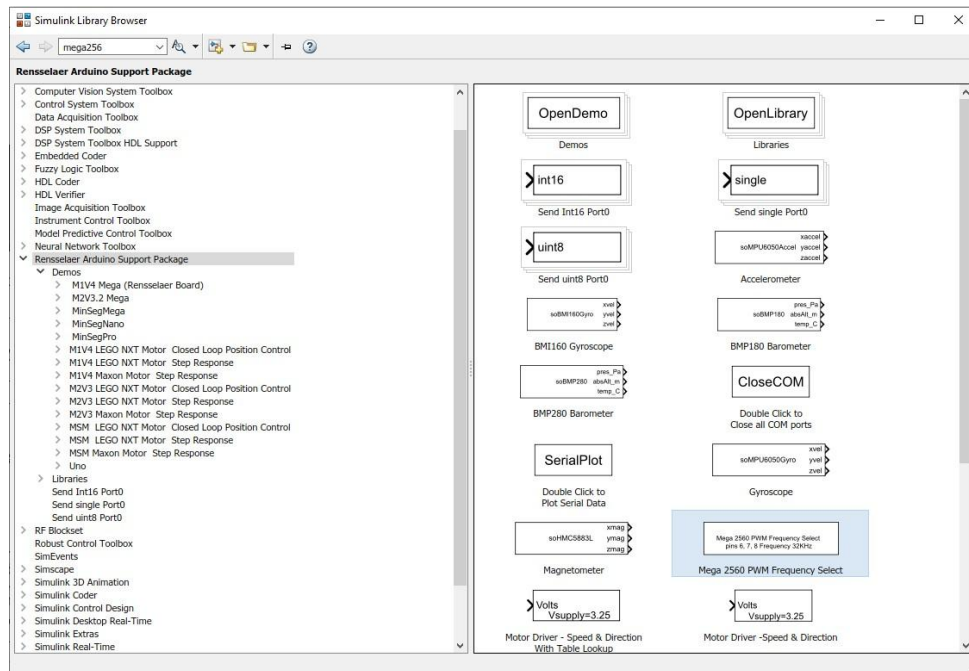


Figure 3.3: Mega2560 PWM Frequency Select” block.

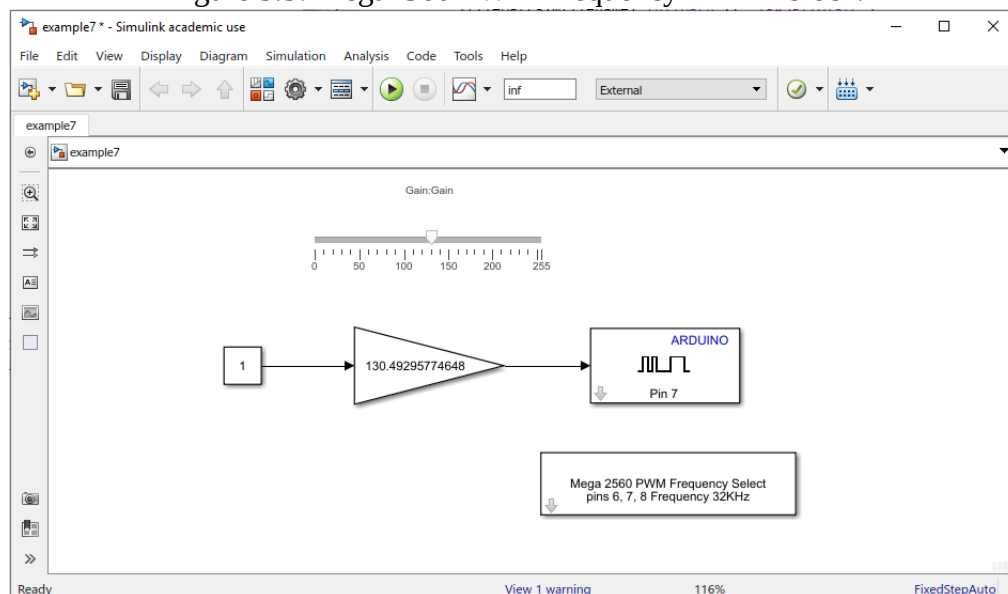


Figure 3.4: Mega2560 PWM Frequency Select” block usage

ACTIVITY 3.3: PWM Motor Drive

The motor used in these experiments is the GM8224S009 DC Servo Gearmotor as shown in Figure 3.5.

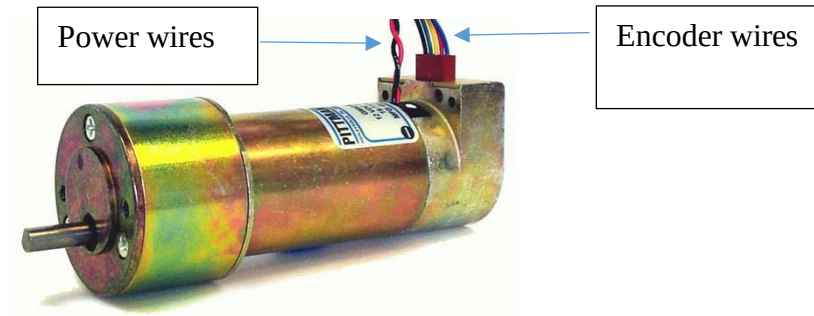


Figure 3.5: GM8224S009 DC Servo Gearmotor

The black (GND) and red (VCC) wires are used for motor drive. **PLEASE BE CAREFUL NOT TO MISTAKE THESE WIRES WITH THE RED/BLACK WIRES OF THE ENCODER.** The encoder can be damaged if you do so since the maximum voltage is 5V for encoder.

The input to the motor can be between -12 to 12V DC. Positive inputs (0-12V) create clockwise rotation while negative inputs (-12-0V) create counter clockwise movement.

Integrated Hewlett-Packard® optical encoders are available on the Series GM8000 motors for position, velocity, and direction feedback (the 5 wires in Figure 3.5), these will be used in Integration Lab 3 to measure angular displacement.

Since the Arduino board cannot generate the required analog output, Pulse Width Modulation (PWM) is used to drive the motor. A dual H-Bridge Motor Driver can be used similar to the one shown in Figure 3.6 (which can drive two motors A and B). For each motor, the board accepts 3 digital inputs (0-5V): (i) Enable to adjust the speed using PWM, (ii) IN1 and (iii) IN2 to drive the motor in forward or reverse mode.

L298N Dual H-Bridge Motor Drive

Setup for Arduino and 12v Battery

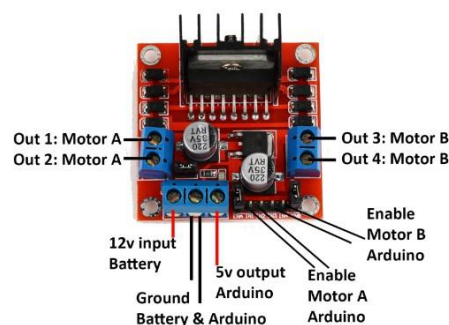


Figure 3.6: A typical H-Bridge circuit.

In this lab you will use the H-bridge driver along with the logic circuit discussed in the course. A snapshot of the circuits discussed are provided below. **TO AVOID DAMAGING THE ENCODER, PLEASE BE CAREFUL NOT TO MISTAKE THE MOTOR POWER TERMINALS WITH THE ENCODER WIRES.**

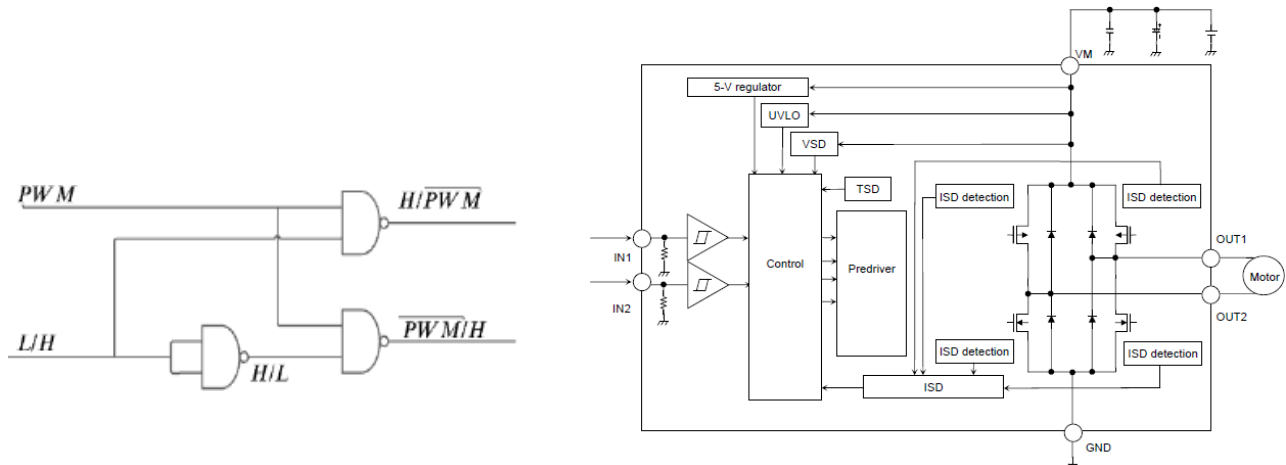


Figure 3.7: PWM driver circuit using the TB6568KQ motor drive chip.

1. Connect the PWM signal (e.g. D9) and a digital output pin of Arduino (e.g., D12, D13, ...) to the L/H pins (see Fig. 3.7) and the output to IN1 and IN2 of the TB6568KQ drive chip.
2. Drag and drop an “Arduino PWM” block from *Simulink Support Package for Arduino Hardware* and specify the PWM pin number in the block parameters (e.g. pin 9). Drag and drop a “digital output” block from *Simulink Support Package for Arduino Hardware* and specify the pin numbers that will be connected to IN1 and IN2 (e.g. D12). See Fig. 3.8.
3. Combine a “constant” connected to an “edit” and a “gain” connected to a “slider” with min and max of -12 and 12 to simulate the analog -12 to 12V input voltage to the motor.
4. Use a “saturation” block to prevent the input signal from being higher than 12V (rated motor voltage).
5. Use an “absolute” value block and a “gain” to map the absolute voltages between 0 to 12V to 0-255 counts (0-100%) to create the PWM signal duty cycle.
6. Use two “compare to constant” blocks from *Simulink>>Logic and Bit Operations* to detect when the signal is positive or negative and activate direction pins (e.g., D12, D13) accordingly.
7. Connect the jumper cables between your H-bridge, the Arduino, the motor and your power supply. Make sure you connect the ground of the power supply and the Arduino board to each other.
8. Use the **Model configuration instructions** Appendix to configure the model to be deployed on the Arduino hardware and run the simulation.
9. Change the slider position and note the motor speed and direction.
10. Use an oscilloscope to see the created square waveform and measure the duty cycle of the PWM output pin and calculate the average output voltage that is created. Use a voltmeter to measure the DC voltage on the output of the H-Bridge and compare it to the values you fed to the system with the slider.

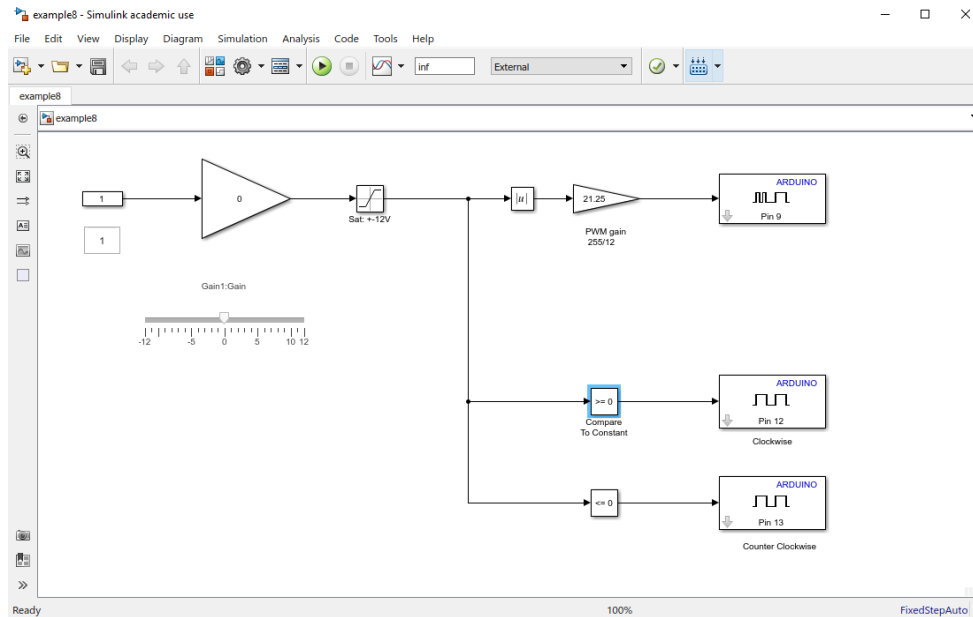


Figure 3.8: PWM Motor Drive

11. Referring to example 3.3, the PWM frequency using pins 4-13 is 976.56 *MMMM*. A motor operating with this PWM frequency will generate an audible noise (20*MMMM* to 20*kkMMMM*). To solve this issue, you can use a block called “Mega 2560 PWM Frequency Select” from “Rensselaer Arduino Support Package”. Copy and paste the block in the same Simulink model subsystem as your PWM block. The block parameters are shown in Fig. 2.9 below.

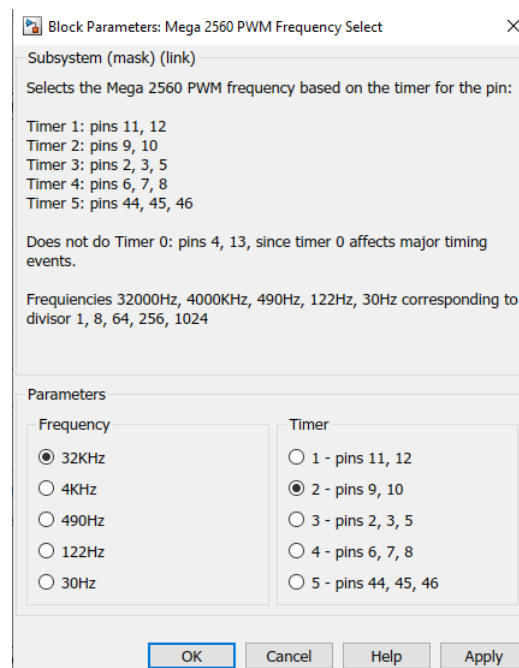


Figure 3.9: PWM Frequency Select

Select the pin you have chosen for PWM on the right hand side and select 32*kkMMMM* as the frequency. Run the model again with these changes and the audio noise should disappear.

Model configuration instructions
(detailed images in example 1.3 (Integration Lab 1))

1. From the dropdown menu by the gear icon on top of the Simulink model, choose the model configuration parameters option.
2. From the configuration parameters window that opens, choose hardware implementation on the left hand side menu and select the Arduino Mega 2560 as the hardware board
3. After choosing the Mega 2560 as your Hardware board, the Target hardware resources option appears at the bottom of the window. From Groups, select Host-board connection and choose the set host COM port as manually and insert the COM port number your Arduino is connected to (e.g., COM 3). Click OK to apply the changes.
4. In the the Solver tab, for the Start time field, enter 0. In the Stop time field, enter the amount of time you want your model to run. For example, enter inf if you want to run the program indefinitely. From the Type list, choose Fixed-step. From the Solver list, choose a solver. For example, ode5 (Dormand-Prince). In the Fixed step size field, enter a sample time. For example, enter 0.01 seconds for a sample rate of 100 samples/second . This sample time should be the same as the sample time in your blocks. It is recommended to keep the sample time constant in all blocks.
5. In the simulink model window, choose the simulation mode as External. Save the model and you are ready to run the simulation.